KPMG

GLOBAL FRAUD DOCUMENT DETECTION (GFDD) TECHNICAL DESIGN DOCUMENT

Version 1.0

Leung, Marco (HK/Trusted)
marco.leung@kpmg.com

Tech Design Document
Global Fraud Document Detection Tool
Version: 1.0
Date: 18 September 2025

Table of Contents

1. Introduction

1.1 Purpose

This Technical Design Document serves as a comprehensive guide for the development, implementation, and maintenance of the Global Fraud Document Detection (GFDD) system. The document aims to provide stakeholders, including developers, managers, and technical teams, with a detailed understanding of the system architecture, methodologies, and technologies employed in the GFDD to deliver effective fraud detection services.

1.2 Scope

The GFDD system is developed to streamline the process of document verification and fraud detection, providing an accessible and user-friendly platform for analyzing diverse document types such as images and PDFs. The scope of this document includes the technical specifications for the tools and algorithms integrated into the platform, covering:

- System architecture and component interactions
- Algorithm descriptions and data processing methodologies
- Technical requirements for software, infrastructure, and security
- Data flow and integration strategies
- Implementation guidelines and best practices

1.3 Objectives

The primary objective of this document is to ensure clarity and alignment across the technical stakeholders involved in the GFDD project. The detailed descriptions and guidelines aim to:

- Facilitate efficient development and deployment of the system
- Ensure robust security and performance standards
- Support effective collaboration and version control practices
- Enable scaling and flexibility for future enhancements

1.4 Audience

This document is intended for backend and frontend developers, system architects, project managers, and other technical personnel responsible for the GFDD system's implementation and maintenance. It serves as a foundation for making informed design decisions and addressing technical challenges.

1.5 Structure

The Technical Design Document is organized into several key sections, each focusing on specific aspects of the GFDD system:

- **System Architecture:** Details the high-level design and interactions of system components.
- **Algorithm Descriptions:** Provides in-depth explanations of the methodologies and calculations used in fraud detection.
- **Technical Requirements:** Outlines the necessary tools, frameworks, and infrastructure for system operation.
- **Data Flow and Integration:** Highlights the movement and processing of data within the system.
- **Implementation Details:** Guides developers on best practices for code structure, development, and deployment.
- **Security Considerations:** Discusses strategies for protecting data and maintaining system integrity.
- **User Roles and Permissions:** Clarifies access control and rights associated with different user roles.

- **Testing and Validation:** Explains procedures for ensuring system reliability and accuracy.

This structured approach allows stakeholders to gain a holistic view of the GFDD system, empowering them to execute their responsibilities with precision and confidence.

2. System Architecture

2.1 Overview

The system architecture for the Global Fraud Document Detection (GFDD) system is designed to efficiently manage document processing, fraud detection, and user interactions through a scalable and robust framework. It combines several components working together to provide comprehensive fraud detection services to users via https://www.gfdd-kpmg.com

2.2 Architecture Components

1. Web Interface
   - **User Interface:** Built using HTML, CSS, and JavaScript, the web interface allows users to upload documents, configure analysis settings, and review results. It aims to provide a seamless experience across various devices.
   - **Interaction Layer:** Powered by Node.js, it manages client-server interactions, handles HTTP requests, and processes user input.
2. Backend Services
   - **Python APIs:** Developed using FastAPI, Python-based APIs serve as the backbone, executing analysis algorithms and returning results to the user interface.
     - AI Integrity Check
     - Pixel & AI Analysis
     - Fraud Visual Tools
     - Text Consistency Checker
3. Database
   - **Data Storage:** Utilizes a database system to store user information, document processing logs, and results. Ensures data durability and accessibility for iterative analysis and user verification.
4. Processing Modules
   - **Algorithm Engines**: Composed of various modules that deploy algorithms for document analysis, fraud detection, and result compilation.
     - **Image Processing**: Manages image enhancements, extracts essential features for analysis.
     - **Machine Learning**: Employs models such as AIorNot for detection purposes.
     - **Text Recognition:** Utilizes OCR to extract and evaluate textual data within documents.

2.3 Interactions

Data Flow

- Upon user input via the web interface, documents are submitted to the backend APIs for processing and evaluation.
- Analysis results are processed by the backend services and stored in the database, enabling efficient retrieval and user review.

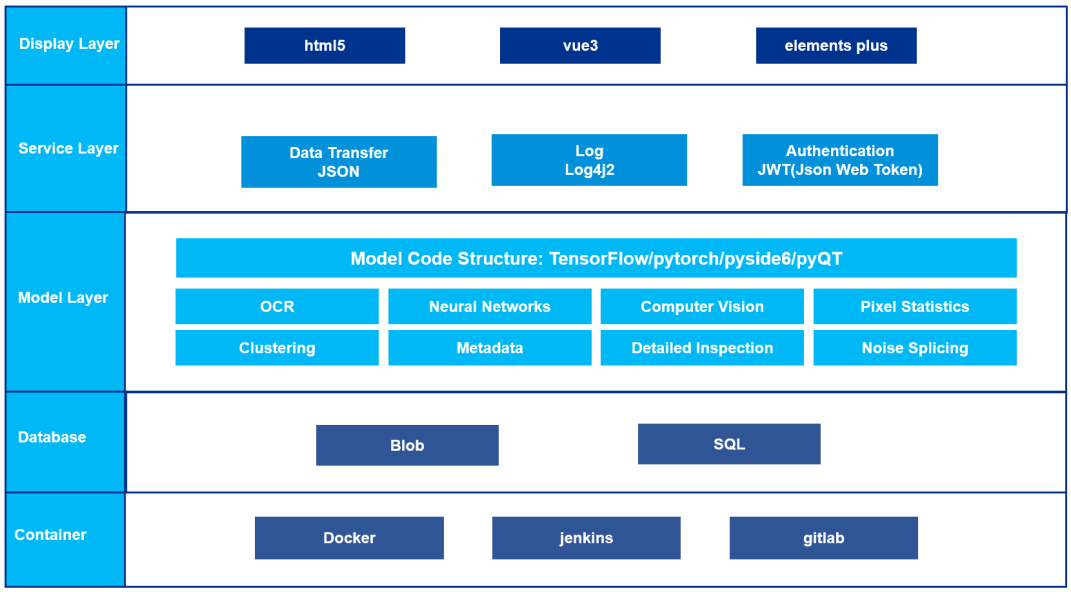
Integration

- External services or third-party APIs for extended functionality or validation, where applicable.
- Continuous deployment updates managed via Git, ensuring smooth integration and collaboration.

2.4 Scalability and Flexibility

- **Containerization:** The system architecture supports Docker and similar technologies for deploying containerized applications, offering flexibility in scaling services as needed.
- **Load Balancing:** Implements load balancing strategies to efficiently handle varying user loads, ensuring consistent performance and availability.

2.5 Diagram



This System Architecture section provides an in-depth understanding of how the GFDD system is structured and interacts, ensuring stakeholders have a clear view of its operations and design principles.

3. Algorithm Description

3.1 AI Integrity Check

3.1.1 Objective

The AI Integrity Check algorithm is designed to assess whether visual content, such as images and videos, has been generated or altered by artificial intelligence. Its primary purpose is to ensure document authenticity by identifying any AI-induced modifications.

3.1.2 Methodology

AIorNot Model

- **Architecture:** Utilizes advanced neural networks, primarily Convolutional Neural Networks (CNNs), to perform detailed analysis of visual data.
- **Training Dataset:** Trained on extensive datasets containing both real photographs and AI-generated images to teach the model to discern unique characteristics between them.
- **Feature Extraction:** Emphasizes identifying peculiarities in pixel-level data, patterning, and illumination characteristics that differ between AI-generated and real images.

Deep Learning Model Implementation

- Utilizes the AIorNot model, trained using deep learning techniques specifically within Convolutional Neural Networks (CNNs).
- Integrates large datasets comprising real and AI-generated images to develop a nuanced understanding of distinguishing features indicative of AI creation.

Neural Network Details

- **Convolutional Layers:** Employs a series of convolutional layers that focus on detecting minute differences in pixel patterns, textures, and lighting conditions present in AI-created imagery versus authentic content.
- **Feature Extraction:** The CNNs analyze high-dimensional data to extract features that are indicative of AI-induced anomalies, such as unnatural texture smoothness or lighting inconsistencies.
- **Classification:** The AIorNot model classifies images based on learned probability scores, with thresholds set to determine if content is more likely AI-generated.

Model Training and Validation

- **Training Process:** Through supervised learning, the AIorNot model uses labeled datasets to refine its accuracy, learning distinct markers of AI versus real content.
- **Validation:** Performance metrics are employed to validate model accuracy and reliability, ensuring robust detection capabilities.

Deep learning allows the AI Integrity Check to provide essential insights into the authenticity and originality of visual content, supporting fraud detection and document validation efforts through sophisticated AI-recognition techniques.

3.1.3 Algorithm Logic

1. **Neural Network Processing**
   - Deploys convolutional layers to evaluate texture and lighting anomalies within the visual data, which are often associated with AI manipulations.
2. **Probability Score Computation**
   - Determines the likelihood of content having an AI origin by applying feature extraction techniques in conjunction with dataset comparisons.

3.1.4 Thresholds

Alerts are triggered when

- **AI Probability:** The computed likelihood of AI-generated content surpasses the established 25% threshold, marking potential tampering.

3.1.5 Alert Triggering Logic

Probability-Based Recognition

- Generates alerts based on probability scores, indicating the likelihood of an image being AI-modified if the real probability falls below 75%

3.1.6 Indicator of Potential Fraud

High AI Probability

- Elevated likelihood scores suggest considerable AI involvement in content creation or alteration, necessitating further investigation.

3.1.7 Output

The AI Integrity Check generates detailed reports that provide insights into the potential AI involvement in document content, including:

- **Probability Scores:** Offers probability metrics indicating the likelihood of AI modification.
- **Classification Results:** Classifies documents based on the probability scores, suggesting authenticity levels and highlighting potential fraud risks.

This exhaustive description equips users with insights into the AI detection processes, ensuring rigorous verification of content authenticity within the GFDD system.

3.2 Pixel & AI Analysis

3.2.1 Objective

The Pixel & AI Analysis suite aims to conduct a detailed evaluation of documents by scrutinizing metadata, pixel statistics, principal components, and potential AI modifications. This suite identifies fraud indicators through a systematic, multidimensional approach.

3.2.2 Algorithm Descriptions

3.2.2.1 Metadata Analysis

Objective

The Metadata Analysis algorithm is designed to uncover potential alterations in the document's life cycle by examining crucial metadata attributes embedded within image files. This process helps identify discrepancies that may indicate tampering or manipulation, contributing to fraud detection efforts.

Methodology

- **Metadata Extraction**
  - Utilizes specialized libraries to extract comprehensive metadata from image files, capturing essential attributes such as creation and modification dates, camera information, and editing tool signatures.
- **Attribute Parsing**
  - Parses extracted metadata fields to identify relevant information, using structured queries to isolate attributes pertinent to document integrity verification.
- **Date and Tool Analysis**
  - Compares creation and modification timestamps to detect anomalies, especially modification dates that appear unusually recent.
  - Searches for metadata identifiers associated with popular editing software to identify potential post-capture modifications.

Details:

- **Creation Date:** Captures the original creation timestamp of the document, serving as a baseline for assessing subsequent changes.
- **Modification Date:** Records the last modified timestamp, critical for identifying unauthorized alterations post-creation.
- **Editing Tools Used:** Detects any editing software signatures embedded within the metadata, which may suggest unsanctioned modifications.

Algorithm Logic

1. **Timestamp Evaluation**

- Analyze chronological coherence between creation and modification dates. Anomalies, such as modification dates preceding creation dates, prompt flags for further scrutiny.

2. **Editing Tool Detection**
   - Employ string matching and regular expression techniques on metadata fields to detect if known editing software has influenced the image, indicating potential tampering.
3. **Modification History Assessment**
   - Examine historical modification records within metadata to identify multiple edits or suspicious modifications aligned with fraud patterns.

Alert Triggering Logic

- **Date Discrepancy Alert**
  - An alert is triggered when modification dates are found to follow creation dates, particularly if discrepancies align with unusual file activity.
- **Editing Software Alert**
  - Activation occurs upon detecting metadata segments indicative of editing tool usage, suggesting possible manipulation or content tampering.

Indicator of Potential Fraud

- **Modification Date Later Than Creation Date:** Suggests unauthorized edits, especially relevant for raw file analysis.
- **Segments Indicating Editing Software:** Points to potential manipulations via external tools.
- **History of Image Modifications:** Multiple modification timestamps indicate repeated edits and probable tampering.

Output

The Metadata Analysis algorithm generates detailed reports on document history and potential alteration indicators, including:

- **Timestamp Analysis:** Offers insights into date-related anomalies that may signal tampering.
- **Editing Tool Identification:** Provides metadata entries identifying editing software signatures.
- **Historical Modification Overview:** Summarizes abnormal modification patterns, aiding in the classification of document authenticity.

This detailed description provides users with a comprehensive understanding of metadata analysis processes, equipping the GFDD system with powerful methodologies for uncovering fraud through metadata scrutiny.

3.2.2.2 Pixel Analysis

Objective

The Pixel Analysis algorithm is designed to assess pixel intensity distributions across images to identify statistical anomalies that may indicate tampering or manipulation, contributing to fraud detection efforts.

Methodology

- **RGB Channel Analysis**
  - Evaluates pixel values in each RGB channel, calculating statistics across different modes (Minimum, Average, Maximum) to detect unusual patterns indicative of alteration.
- **Threshold Evaluation**
  - Utilizes predefined percentage thresholds for each mode to determine normalcy, triggering alerts for values exceeding these benchmarks.

- **Standard Deviation and Distribution Assessment**
  - Calculates the standard deviation to measure pixel intensity variability, while analyzing distribution patterns to identify bimodal or skewed characteristics.

Details

- **Minimum Pixel Intensity:** Identifies the lowest pixel value in the image, suggesting areas that may be underexposed.
- **Average Pixel Intensity:** Represents the mean pixel value, providing insights into the overall brightness level.
- **Maximum Pixel Intensity:** Determines the highest pixel value, indicating regions potentially dominant by a single color.
- **Standard Deviation:** Measures the variability or dispersion of pixel intensities around the average value.
- **Distribution:** Evaluates the shape of pixel intensity distribution, such as normal, skewed, or bimodal.

Algorithm Logic

1. **Channel Comparison**
   - Analyzes RGB values by comparing each channel's intensity to determine the mode-specific pixel count for Minimum, Average, and Maximum calculations.
2. **Threshold Breach Detection**
   - Computes the percentage of pixels within each mode that meets or surpasses set thresholds, triggering alerts for breaches.
3. **Statistical Variability Assessment**
   - Evaluates standard deviation for signs of abnormal pixel variability suggesting manipulation, alongside detecting multi-modal peaks in histograms for further analysis.

Thresholds

- **Alerts arise when**
  - **Minimum Mode:** Exceeds a 28% threshold, suggesting underexposure.
  - **Average Mode:** Surpasses a 30% threshold, indicating abnormal brightness levels.
  - **Maximum Mode:** Exceeds a 38% threshold, pointing to single-color dominance.
  - **Standard Deviation:** Outliers in variability signal possible tampering.
  - **Bimodal Distribution:** Detecting multiple peaks triggers alerts as it indicates diverse manipulation patterns across regions.

Alert Triggering Logic

- **Threshold Breach:** Alerts are generated when calculated RGB proportions exceed mode-specific thresholds.
- **Anomalous Standard Deviation:** Spike in standard deviation beyond set limits indicates potential tampering.
- **Distribution Analysis Alert:** Multi-modal peaks in intensity histogram suggest manipulation.

Indicators of Potential Fraud

- **Sudden Spike in Standard Deviation:** A significant increase in standard deviation suggests unusual variations in pixel intensities, pointing to tampering.
- **Abnormal Average Pixel Intensity:** Significant deviations from expected values indicate potential alterations in image parts.

- **Bimodal Distribution:** Multi-modal peaks suggest different regions of the image have been manipulated differently.
- **Outliers in Minimum or Maximum Values:** Extreme values not aligning with the rest of the image suggest tampering.

Output

The Pixel Analysis algorithm produces detailed reports focusing on pixel intensity characteristics and anomalies, providing:

- **Pixel Intensity Profiles**: Highlights areas of interest based on statistics.
- **Distribution Maps**: Visual representation of pixel distribution findings.
- **Indicator Summary**: Provides a summary of identified anomalies, aiding in fraud detection.

This comprehensive description equips users with a deep understanding of the pixel analysis processes, ensuring meticulous examination of potential image manipulations within the GFDD system.

3.2.2.3 Principal Component Analysis (PCA)

Objective

The Principal Component Analysis (PCA) algorithm is designed to project pixel values onto salient components to identify patterns and anomalies within images. This analysis helps detect unusual variations that may indicate tampering or manipulation, contributing to fraud detection efforts.

Methodology

- **PCA Decomposition:**
  - Utilizes statistical techniques to compute eigenvectors and eigenvalues, capturing dominant patterns and variances within the image data.
- **Distance and Projection Analysis:**
  - Measures orthogonal distances and projection similarities along principal components to detect pixel-level anomalies and assess content pattern shifts.

Details

- **Modes**
  - **Distance Mode:** Evaluates orthogonal distance from principal component vectors, useful for identifying pixel-level anomalies like fake pixels or sudden color changes.
  - **Projection Mode:** Measures similarity along principal component vectors, assessing manipulated region structures.
- **Components:**
  - **Dominant Pattern (#1):** Represents the primary color and variance pattern.
  - **Secondary Pattern (#2):** Captures alternative patterns contributing to variation.

Algorithm Logic

1. **Eigenvalue and Eigenvector Calculation**

- Computes eigenvalues and eigenvectors to determine variance and directionality of color patterns, analyzing deviations from expected norms.

2. **Distance Map Analysis**

- Detects inconsistencies in how color pixels align with principal components; fake documents often introduce irregularities creating inflated distances between RGB pixels and PCA direction vectors.

3. **Projection Map Analysis**

- Real documents typically exhibit diverse pixel patterns, while flatness in projection space may suggest copy-paste or synthetic areas.

Thresholds

- **Alerts arise when:**
  - **Mean Vector Values:** Fall outside the 0 to 255 range, indicating potential alterations that affect pixel color summaries.
  - **Eigenvector Norms:** Exceed the normative L2 norm range [0.85, 1.15], suggesting unusual alignment and direction, indicative of manipulation.
  - **Eigenvalue Distribution:** Detected variances fall outside expected limits, pointing to altered regions within the image.

Alert Triggering Logic

- **Eigenvalue Variance Alert**
  - Triggered by high variance in principal components, indicating possible manipulation.
- **Eigenvector Direction Alert**
  - Activated when unexpected eigenvector directions suggest image manipulation.
- **Mean Vector Anomalies Alert**
  - Alerts occur when mean vector values fall outside typical ranges, suggesting alterations.

Indicators of Potential Fraud

- **High Variance in Principal Components**
  - Unusually high or low eigenvalues suggest manipulation, such as altered regions in the image.
- **Unusual Eigenvector Directions**
  - Non-aligned eigenvectors indicate potential tampering, diverging from typical color variations seen in natural images.
- **Mean Vector Anomalies**
  - Abnormal mean values can signal alterations, especially if they fall outside expected ranges for a given image type.

Output

The PCA algorithm produces detailed analyses highlighting principal component anomalies, including:

- **Variance and Direction Maps:** Displays eigenvalue and eigenvector distributions.
- **Distance and Projection Profiles:** Visualizes how color pixels align and deviate from principal components.
- **Fraud Indicator Summary:** Highlights potential anomalies, aiding in fraud detection.

This thorough description equips users with insights into principal component analysis processes, ensuring comprehensive evaluation of potential image manipulations within the GFDD system.

3.2.2.4 AI Detection

Objective

The AI Detection algorithm aims to ascertain whether visual content, such as images, has been generated or altered by artificial intelligence, ensuring document authenticity and originality.

Methodology

- **Machine Learning Model Implementation**

- Uses the AIorNot deep learning model, integrating Convolutional Neural Networks (CNNs) trained on extensive datasets of both real and AI-generated images.
- Extracts features that distinguish AI-induced anomalies from genuine images based on pixel patterns, textures, and lighting conditions.

Details

- **AIorNot Model**
  - Employs advanced CNN layers to analyze high-dimensional data, learning characteristic differences between AI-created content and real photographs.
- **Feature Extraction and Classification**
  - Focuses on identifying minute and distinct features in pixel patterns that are indicative of AI involvement.

Algorithm Logic

- **Neural Network Processing:**
  - Applies convolutional layers to detect deviations in texture and lighting conditions, suggestive of AI manipulation.
- **Probability Score Computation**
  - Calculates likelihood scores that the image has an AI origin based on learned features and comparisons with training data.

Thresholds

- **Alerts are triggered when:**
  - **AI Probability:** The likelihood of AI-generated content exceeds 25%, indicating enhanced potential for tampering.

Alert Triggering Logic

- **Probability-Based Recognition:**
  - Generates alerts based on computed probability scores signifying the likelihood of the image being AI-modified. If real probability falls below the threshold of 75%, AI alerts are triggered.

Indicators of Potential Fraud

- **High AI Probability:** Elevated scores suggest a considerable chance of AI involvement in creating or altering the image, warranting further scrutiny.

Output

The AI Detection algorithm generates detailed reports providing insights into potential AI involvement, including:

- **Probability Scores:** Offers a metric indicating the likelihood of AI modification.
- **Classification Summary:** Suggests authenticity levels, emphasizing potential fraud risks.

This detailed description enables users to understand the AI detection processes, ensuring authentic and original content verification within the GFDD system.

3.2.3 Output

Each algorithm contributes to crafting a comprehensive report that outlines specific alert conditions and corresponding fraud risk metrics. This enables a meticulous assessment of document authenticity, empowering users with robust tools for fraud detection and prevention.

3.3 Fraud Visual Tools

3.3.1 Objective

The Fraud Visual Tools suite is designed to provide a comprehensive visual assessment of documents, enabling detailed analysis to uncover indications of tampering or manipulation. By using advanced image analysis techniques such as Edge Detection, Error Level Analysis (ELA), and Copy-Move Forgery Detection, it facilitates the identification of potentially fraudulent activities. These tools employ visualization methodologies to highlight suspect areas, offering crucial insights into document authenticity and integrity.

3.3.2 Algorithm Descriptions

3.3.2.1 Edge Detection

Objective

The Edge Detection algorithm identifies and extracts edges within an image by detecting significant changes in intensity. This helps to visualize and assess potential areas of tampering, highlighting boundaries and transitions that may suggest modifications.

Methodology

The algorithm employs an optimized Sobel edge detection technique, enhanced by several refinements to improve precision and robustness:

- **Kernel Usage:** The Sobel operator is used for calculating image derivatives. For a small kernel size (3x3), which is typical for detecting finer details, the Scharr operator is leveraged for higher accuracy. A Gaussian blur is applied for larger kernels to reduce noise.
- **Gradient Calculation:** Calculates the gradient magnitude by computing the Euclidean distance using the derivatives in the x and y directions (sobel_x and sobel_y). This is essential for determining the edge strength at each pixel.
- **Threshold Filtering:** To minimize noise, gradients below a certain threshold are disregarded. This threshold is scaled relative to the maximum gradient magnitude within the image.
- **Normalization and Contrast Enhancement:** The detected edges are normalized to a range of 0 to 255 and enhanced for visibility using contrast stretching. A lookup table (LUT) is applied to adjust the contrast based on the user-defined settings.

Features

- **Grayscale Conversion:** Allows optional conversion of output edge images to grayscale to simplify visualization.
- **Anti-Forensics Detection:** New functionality that detects signs of image smoothing or blurring intended to hide evidence of manipulation. This feature can increase edge detection sensitivity in the presence of such artifacts.

Processing Steps

1. **Image Input and Validation**
   - The algorithm reads the input image, validates its integrity, and checks for pre-processing anti-forensic measures.
2. **Kernel Configuration**
   - The kernel size is determined by user-defined radius, influencing the granularity of edge detection.
3. **Edge Detection Execution**
   - For each image channel, apply the optimized Sobel operator to extract edge information.

- Aggregate results across color channels to construct the final edge detection image.

4. **Performance and Parameter Logging**
   - Records processing time, image dimensions, and applied settings for transparency and reproducibility.
   - Generates forensic warnings if anti-forensic measures are detected.
5. **Result Compilation**
   - Returns processed images with edge information, suitable for visual inspection, and logs key parameters and any forensic warnings for further analysis.

Through this approach, the Edge Detection algorithm offers a robust toolset for visually evaluating documents to identify potential tampering, ensuring meticulous examination of image boundaries and transitions.

3.3.2.2 Error Level Analysis (ELA)

Objective

Error Level Analysis (ELA) is designed to assess the authenticity and integrity of digital images by highlighting quality variations across different compression levels. This method helps to visualize potential areas of tampering or manipulation within an image.

Methodology

The ELA algorithm leverages image compression as a tool to identify discrepancies that may indicate amendments. By recompressing an image using varying JPEG quality settings, ELA examines the differences in image quality between the original and the compressed version.

- **Compression and Difference Calculation:** Compresses the image using JPEG with a specified quality setting, then calculates the absolute difference between the original and compressed versions to reveal tampering footprints.
- **Scaling and Enhancement:** The resulting difference is scaled using a factor to adjust the visual emphasis on these discrepancies, allowing for enhanced identification of potential tampered regions.
- **Contrast Adjustment**: A lookup table (LUT) is applied to adjust the contrast of the ELA output, improving visibility of differences and aiding in interpretation.
- **Grayscale Conversion:** Allows optional conversion of the output to grayscale, which can simplify the detection process by focusing on intensity variations.

Processing Steps

1. **Image Input and Validation:** Reads and validates the input image, ensuring it is suitable for processing.
2. **JPEG Compression:** Utilizes a specified quality setting for JPEG compression to generate a basis for comparison.
3. **Difference Calculation:** Computes absolute pixel differences between the original and compressed image, transforming these values to highlight potential tampering.
4. **Contrast Enhancement:** Adjusts the contrast to accentuate visible differences, making potential tampered regions more discernible.
5. **Finalization and Metadata Logging:** Compiles metadata and encodes the processed image for output, detailing processing time, quality settings, contrast adjustments, and any other applied parameters for transparency and reproducibility.

Through this approach, the Error Level Analysis algorithm provides a powerful visualization tool to evaluate documents for signs of tampering, ensuring meticulous inspection of image inconsistencies.

3.3.2.3 Copy-Move Forgery

Objective

The Copy-Move Forgery Detection algorithm aims to identify areas within an image where regions have been copied and pasted, potentially to conceal modifications. This technique leverages key point detection and matching to pinpoint duplications, which can indicate image manipulation.

Methodology

The algorithm implements a structured approach to detect and visualize duplicated areas using various key point detection and matching techniques:

- **Key Point Detection**
  Utilizes feature detectors such as BRISK, ORB, or AKAZE to identify and describe key points within the image. These detectors are chosen based on user preference, each offering trade-offs between speed and robustness.
- **Response Filtering**
  Filters key points based on their response values. Key points with strong responses, indicative of prominent features, are selected for further processing to reduce noise.
- **Descriptor Matching**
  Employs a brute-force matcher with Hamming distance to find matching key points between duplicate regions. A radius matching approach is utilized to detect potential copy-move areas.
- **Cluster Formation**
  Groups matches into clusters based on geometric and similarity constraints, ensuring that collected matches form coherent groups that suggest duplication.
- **Visualization and Analysis**
  Visually represents detected matches and clusters on the image, enabling intuitive assessment of forgery. Also measures angles and distances to interpret potential areas of manipulation.

Processing Steps

1. **Image Input and Preprocessing:** Loads the input image and performs any necessary preprocessing, such as applying masking for focused analysis or detecting anti-forensic measures.
2. **Key Point Extraction:** Detects key points using the specified detector and filters results based on response thresholds to ensure quality matches.
3. **Match Pairing:** Matches descriptors using a radius matcher, filtering matches based on distance constraints to identify genuine duplications.
4. **Cluster Evaluation:** Forms clusters of matches by examining geometric alignment and similarity, identifying potential copy-move operations.
5. **Result Visualization:** Outputs a marked-up image highlighting detected key points, matches, and lines between duplicated regions for in-depth visual analysis.
6. **Performance and Warnings:** Records processing time, key metrics, and any anti-forensic warnings for transparency and further evaluation.

Through this method, the Copy-Move Forgery Detection algorithm provides a powerful visual tool to analyze documents for signs of duplication and tampering, ensuring careful examination of potential image forgery.

3.3 Output

Each tool within the Fraud Visual Tools suite generates enhanced visual outputs that highlight areas of interest for further examination. The results typically include:

- **Edge Maps:** Depict areas of sharp intensity changes to reveal potential tampered regions.
- **ELA Results:** Contrast-adjusted images showing quality variations suggestive of underlying alterations.
- **Forgery Visualization:** Annotated images with key points and lines connecting duplicated regions indicative of copy-move forgery.

This suite delivers a robust set of visualization tools that bolster document examination procedures, aiding in the detection of fraudulent modifications through clear and insightful image representations.

3.4 Text Consistency Checker

3.4.1 Objective

The Text Consistency Checker aims to verify the integrity of textual information within documents. By utilizing advanced Optical Character Recognition (OCR) and textual analysis, it ensures consistency and accuracy in document content, particularly focusing on financial statements such as invoices and bank statements.

3.4.2 Methodology

The algorithm employs OCR for extracting textual content from documents and subsequently analyzes this data to identify inconsistencies and potential manipulations in textual or numerical information

3.4.3 Algorithm Logic

1. **Image Preprocessing**
   - **Enhancement:** Converts uploaded images to grayscale and enhances low-contrast areas using CLAHE (Contrast Limited Adaptive Histogram Equalization) to optimize OCR accuracy.
   - **Resolution Optimization:** Adjusts image DPI to a target level (e.g., 200 DPI) to improve text recognition performance.
2. **Text Extraction**
   - Uses Tesseract OCR to extract text from enhanced images, with multi-language support for recognizing both Chinese and English text.
   - Configures OCR settings based on document type and expected text layout using custom configurations to optimize recognition accuracy.
3. **Data Extraction**
   - **Amounts and Dates:** Extracts numerical values and dates from recognized text for analysis. Regular expressions are employed to identify patterns within the text that align with expected formats.
   - **Description and Sequences:** Captures descriptive text and validates numerical sequences to ensure logical consistency.
4. **Data Validation**
   - **Amount Validation:** Checks sequence and integrity of financial figures, identifying discrepancies in expected cumulative values or account balances.
   - **Date Alignment:** Verifies that transaction dates align with statement periods, flagging any outliers or inconsistencies.
   - **Templates:** Leverages predefined templates to handle specific document structures tailored to different banking statements, enabling targeted extraction and validation.
5. **Result Compilation**

- Returns extracted text and alerts if inconsistencies are detected, in a structured format suitable for user review and further analysis.

3.4.4 Alert Triggering Logic

- **Inconsistent Amounts:** Alerts are triggered when identified discrepancies in amounts exceed predefined error thresholds (e.g., a 10% error rate).
- **Misaligned Dates:** Generates alerts for transaction dates not within specified statement periods or context.
- **Potential Fraud Classification:** If any alerts are triggered, the document may be classified as potential fraud for further scrutiny.

3.4.5 Output

The Text Consistency Checker produces comprehensive reports detailing extracted texts and any detected inconsistencies. These outputs include:

- **Extracted Text and Preview:** Provides recognized content and clarity on document structure.
- **Alert Messages:** Highlights potential issues in amounts, dates, or descriptions, with specific details to aid in resolution.
- **Fraud Indicator:** Classifies the likelihood of fraud based on validated data integrity and coherence.

This algorithm facilitates thorough examination of textual content within documents, supporting effective fraud detection and assurance of document integrity.

4. Data Flow and Integration

4.1 Overview

This section outlines the data flow and integration points within the Global Fraud Document Detection (GFDD) system. It highlights how data is ingested, processed, and output, alongside the interactions between internal components and external systems.

4.2 Data Input

Sources

- User Uploads: Documents (images, PDFs) are uploaded by users through a web interface.
- Supported Formats: JPEG, PNG, TIFF, HEIC, HEIF, PDF, accommodating diverse use cases.
- Batch Processing: Allows simultaneous processing of multiple documents to enhance efficiency.

Supporting Image Format	Document Format
1. Windows bitmaps - *.bmp, *.dib 2. GIF files - *.gif 3. JPEG files - *.jpeg, *.jpg, *.jpe 4. JPEG 2000 files - *.jp2 5. Portable Network Graphics - *.png 6. WebP - *.webp 7. AVIF - *.avif 8. Portable image format - *.pbm, *.pgm, *.ppm, *.pxm, *.pnm 9. PFM files - *.pfm 10. Sun rasters - *.sr, *.ras 11. TIFF files - *.tiff, *.tif 12. OpenEXR Image files - *.exr\	1. Portable Document Format - *.pdf

13. Radiance HDR - *.hdr, *.pic	

Limitations

- **File Count:** Up to 20 files can be uploaded per batch, ensuring efficient processing while maintaining system stability.
- **File Size:** Each uploaded file is limited to a maximum size of 10MB, allowing quick upload and processing times without overloading server resources.

Flow

- Upon upload, documents are passed to the respective analysis modules (e.g., AI Integrity Check, Pixel & AI Analysis) for preprocessing and examination.

4.3 Processing Flow

Pipeline Steps:

1. **Preprocessing:**
   - **Metadata Extraction:** Captures essential metadata attributes for initial validation.
   - **Image Enhancement:** Applies techniques such as CLAHE for optimal OCR performance.
2. **Core Analysis:**
   - **AI Integrity Check:** Detects AI modifications using deep learning models.
   - **Pixel & AI Analysis:** Conducts in-depth pixel examination and PCA for anomaly detection.
   - **Fraud Visual Tools:** Employs edge detection, ELA, and copy-move forgery detection for visual inspection.
   - **Text Consistency Checker:** Extracts and validates textual information for consistency.

Integration Points:

- Data processed by different modules is cross-referenced to provide comprehensive fraud detection insights.
- Alerts and flags generated during processing are unified to produce a cohesive evaluation report.

4.4 Data Output

Results Compilation:

- **Alerts and Reports:** Offers detailed metrics and classifications, specifying the probability of fraud and authenticity.
- **Export Options:** Provides outputs in various formats (CSV, annotated images) for user review and archiving.
- **User Feedback:** Displays concise summaries of findings to users through the web interface for immediate action.

4.5 External Systems

Integration Options:

- Potential connections with external databases or APIs for enhanced data validation and cross-referencing.
- Support for integrating with third-party monitoring tools, enabling ongoing surveillance of document authenticity.

4.6 Diagram

Visual Representation:

- **Data Flow Diagram:** A schematic illustrating the interactions between data inputs, processing modules, and outputs, highlighting integration points and dependencies.

By documenting the data flow and integration architecture, this section facilitates a clear understanding of how data traverses the GFDD system, enabling efficient processing and providing actionable fraud detection results.

5. Technical Requirement

5.1 General Requirements

The GFDD system is hosted and operated through its dedicated website (https://www.gfdd-kpmg.com), providing users with access to its comprehensive fraud detection tools via a web interface. The infrastructure supporting this system includes essential technologies and practices required for optimal performance and maintenance.

5.2 Software Requirements

Languages and Frameworks

- **Python 3.9.13:** Serves as the primary language for backend development, utilized for implementing algorithms such as AI Integrity Check, Pixel & AI Analysis, Fraud Visual Tools, and Text Consistency Checker.
  - Key Libraries: OpenCV, NumPy, TensorFlow, FastAPI.
- Node.js 16: Facilitates front-end server operations, handling user requests, and enhancing the website's dynamic response capabilities.
  - Key Libraries: Express, Axios.

Version Control

- **Git:** Employed for managing code versions and collaborative updates, ensuring a structured approach to development, integration, and deployment.

5.3 Deployment and Hosting

Website Hosting

- The GFDD system is hosted under the domain https://www.gfdd-kpmg.com, ensuring accessibility for end-users to leverage fraud detection features without requiring local infrastructure.

5.4 Compatibility Considerations

Cross-Platform Operation

- Ensure browser compatibility to support user access across various devices and operating systems, maintaining consistent user experience and functionality.

5.5 Performance and Optimization

Resource Utilization

- The website handles user interactions and data processing efficiency without specific hardware requirements, leveraging cloud-based services and APIs where applicable.

These technical requirements establish the foundational software and operational components necessary for maintaining and running the GFDD system effectively, enabling continuous fraud detection services to users via its website interface.

6. Implementation Details

6.1 Core Structure

6.1.1 Backend (Python)

- **Main Framework:** The backend is primarily built using FastAPI for creating robust APIs that facilitate interactions with the fraud detection algorithms.
  - **Algorithms Directory:** Contains modules for AI Integrity Check, Pixel & AI Analysis, Fraud Visual Tools, and Text Consistency Checker.
  - **Utilities:** Hosts helper functions and classes for image processing, data handling, and common operations.
  - **API Endpoints:** Defines routes for receiving user uploads, processing data, and returning results through JSON responses.
  - **Configuration:** Includes settings and environment variables for customizing runtime parameters and external integrations.

6.1.2 Frontend (Node.js)

- **Main Framework**: Utilizes Express.js to serve the web application, handle user requests, and deliver dynamic content.
  - **Public Assets:** Stores static files (HTML, CSS, JavaScript) that define the user interface and interaction models.
  - **Views:** Contains template files for rendering client-side pages and components.
  - **Middleware:** Implements security, logging, and session management functions to streamline user interactions and maintain robust application performance.

6.2 Key Considerations

Scalability

- Ensure the application is capable of handling increasing user demand by optimizing API endpoints and utilizing cloud infrastructure when necessary.

Performance Optimization

- Optimize image processing algorithms by leveraging efficient libraries like OpenCV and NumPy.
- Adopt caching strategies where applicable to reduce redundant computations and enhance response times.

Security

- Implement authentication mechanisms to safeguard user data and control access to sensitive operations.
- Regularly update dependencies to mitigate vulnerabilities and enhance system security.

Maintainability

- Use Git for version control to facilitate collaborative development and track changes effectively.
- Encourage modular coding practices to maintain a clean and extendable codebase that adapts to future requirements.

6.3 Development Environment

Local Setup

- Setup instructions for running the application locally, including dependencies installation, environment variable configuration, and initial deployment steps.

Testing and Debugging

- Recommend testing suites for validating algorithm accuracy and system robustness.
- Establish best practices for debugging and error handling to ensure smooth issue resolution.

This section provides crucial implementation information that aids developers in understanding the organization, practices, and strategies embedded within the GFDD codebase, ensuring effective development and maintenance of the system.

7. Security Consideration

7.1 Data Security

Encryption

- Implement Secure Socket Layer (SSL) certificates to encrypt data transmitted between the user and server, ensuring confidentiality and protection against eavesdropping.
- Utilize encryption for sensitive data at rest, such as user-uploaded documents, to safeguard them from unauthorized access.

Access Controls

- Enforce role-based access control (RBAC) within the application to restrict permissions and ensure users have the appropriate level of access for their role.
- Require user authentication for accessing sensitive operations or data, utilizing secure authentication protocols such as OAuth2 or JWT.

7.2 Infrastructure Security

Network Security

- Employ firewalls to filter traffic and prevent unauthorized access to system resources.
- Regularly update server configurations and software packages to close vulnerabilities and maintain security integrity.

Environment Isolation

- Deploy the application using containerization tools (e.g., Docker) to isolate environments and manage dependencies within controlled boundaries.
- Use virtual private networks (VPNs) for secure remote access when necessary.

7.3 Application Security

Input Validation

- Validate all user inputs thoroughly to prevent injection attacks, ensuring any data processed is sanitized and meets expected formats.
- Enforce strict file size and format checks on uploaded documents to mitigate potential security risks associated with large or malformed files.

Dependency Management

- Regularly audit third-party libraries and dependencies for vulnerabilities, considering updates and patches to maintain strong security posture.
- Employ automated tools to scan for known vulnerabilities and apply fixes promptly when identified.

7.4 Monitoring and Response

Logging and Monitoring

- Enable comprehensive logging of user activities and system events to detect anomalies and unauthorized access attempts.
- Use monitoring tools to actively observe system behavior and performance, providing alerts for suspicious activities.

Incident Response

- Develop an incident response plan to address security breaches effectively, outlining steps for containment, eradication, and recovery.
- Conduct regular security drills to ensure the team is prepared for potential threats and can respond swiftly.

8. User Roles and Permissions

8.1 Overview

The GFDD website operates with distinct user roles to ensure efficient access control and management. This section outlines the responsibilities and permissions associated with each role.

8.2 Roles and Descriptions

Admin

- **Responsibilities**
  - Manage user accounts, including the creation and deletion of user profiles.
  - Assign administrative rights to other users, facilitating role-based access control.
- **Permissions**
  - Full access to all tools and data.
  - Ability to configure system settings and manage user access permissions.

User

- **Responsibilities**
  - Utilize the GFDD tools provided by the website for document analysis and fraud detection.
- **Permissions**
  - Access to tools for document processing but restricted from user management tasks or system configurations.

Developer

- **Responsibilities**
  - Engage in active development and maintenance tasks for both frontend and backend components.
  - Record and manage updates using Git for version control.
- **Permissions**

- Access to development environment repositories in Git.
- Backend developers have access to server-side code; frontend developers handle user interface and experience code.

8.3 Role Management

Implementation

- Role assignments are managed through the admin interface, with secure authentication protocols ensuring appropriate access.
- Logging mechanisms track role changes to maintain accountability and transparency.

Role-based access control ensures that the GFDD website operates smoothly and securely, with each user group having defined responsibilities and permissions to support organizational goals and system integrity.

9. Testing and Validation

9.1 Testing Strategies

Automated Testing

- **Unit Testing:** Implement comprehensive unit tests to validate each function or module in isolation, ensuring individual components perform as expected.
- **Integration Testing:** Conduct integration tests to verify that different modules interact correctly and produce accurate results when combined. This is particularly important for algorithms that work in conjunction with each other.
- **Regression Testing:** Regularly run regression tests to ensure new code changes do not adversely affect existing functionalities, preserving system stability.

Manual Testing

- **User Acceptance Testing (UAT):** Collaborate with end-users to test the system in real-world scenarios, confirming that it meets their requirements and expectations.
- **Exploratory Testing:** Encourage testers to perform exploratory testing, identifying edge cases and unexpected behaviors by simulating varied user interactions.

9.2 Validation Criteria

Accuracy and Performance

- **Algorithm Validation:** Validate the accuracy of each algorithm, such as AI Integrity Check, Pixel & AI Analysis, and Fraud Visual Tools. Compare results against known datasets to ensure detection capabilities.
- **Performance Metrics:** Measure system performance under different load conditions, including processing speed, response time, and resource usage.

Security Tests

- **Security Validation:** Conduct penetration testing and vulnerability scans to verify that the system is secure from unauthorized access and threats.
- **Role-Based Access Control:** Ensure role-based permissions function correctly, granting appropriate access levels to admins, users, and developers.

User Interface Testing

- **Interface Consistency:** Test the website's user interface to ensure that it is consistent, intuitive, and accessible across various browsers and devices.
- **Error Handling and Feedback:** Validate that user-facing errors are handled gracefully, providing clear and informative feedback for resolution.

Data Integrity and Handling

- **Data Accuracy:** Verify that data input (e.g., documents) is processed accurately, with results reflecting correct analysis outputs.
- **File Uploads and Storage:** Test file upload mechanisms to ensure they handle various formats and sizes within specified limits while safeguarding data security.

9.3 Reporting and Optimization

Test Reporting

- Document and report testing results, including pass/fail status, identified bugs, and suggestions for improvements.
- Maintain a comprehensive testing log to track issues and progress over time.

Continuous Improvement

- Incorporate feedback from testing cycles into development practices, optimizing algorithms and system functionalities for enhanced performance and reliability.

This Testing and Validation section outlines the strategies and criteria essential for ensuring the GFDD system's accuracy, security, and user satisfaction, supporting ongoing system improvement and maintenance efforts.